

Mini RFC: Error Handling

April 21, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

An endpoint that detects an error **SHOULD** signal the existence of that error to its peer. Both transport-level and application-level errors can affect an entire connection. Only application-level errors can be isolated to a single stream.

The most appropriate error code **SHOULD** be included in the frame that signals the error. Where this specification identifies error conditions, it also identifies the error code that is used; though these are worded as requirements, different implementation strategies might lead to different errors being reported. In particular, an endpoint **MAY** use any applicable error code when it detects an error condition; a generic error code (such as `PROTOCOL_VIOLATION` or `INTERNAL_ERROR`) can always be used in place of specific error codes.

A stateless reset is not suitable for any error that can be signaled with a `CONNECTION_CLOSE` or `RESET_STREAM` frame. A stateless reset **MUST NOT** be used by an endpoint that has the state necessary to send a frame on the connection.

2 Connection Errors

Errors that result in the connection being unusable, such as an obvious violation of protocol semantics or corruption of state that affects an entire connection, **MUST** be signaled using a `CONNECTION_CLOSE` frame.

Application-specific protocol errors are signaled using the `CONNECTION_CLOSE` frame with a frame type of `0x1d`. Errors that are specific to the transport, including all those described in this document, are carried in the `CONNECTION_CLOSE` frame with a frame type of `0x1c`.

A `CONNECTION_CLOSE` frame could be sent in a packet that is lost. An endpoint **SHOULD** be prepared to retransmit a packet containing a `CONNECTION_CLOSE` frame if it receives more packets on a terminated connection. Limiting the number of retransmissions and the time over which this final packet is sent limits the effort expended on terminated connections.

An endpoint that chooses not to retransmit packets containing a `CONNECTION_CLOSE` frame risks a peer missing the first such packet. The only mechanism available to an endpoint that continues to receive data for a terminated connection is to attempt the stateless reset process.

As the AEAD for Initial packets does not provide strong authentication, an endpoint **MAY** discard an invalid Initial packet. Discarding an Initial packet is permitted even where this specification otherwise mandates a connection error. An endpoint can only discard a packet if it does not process the frames in the packet or reverts the effects of any processing. Discarding invalid Initial packets might be used to reduce exposure to denial of service.

3 Stream Errors

If an application-level error affects a single stream but otherwise leaves the connection in a recoverable state, the endpoint can send a `RESET_STREAM` frame with an appropriate error code to terminate just the affected stream.

Resetting a stream without the involvement of the application protocol could cause the application protocol to enter an unrecoverable state. `RESET_STREAM` **MUST** only be instigated by the application protocol that uses QUIC.

The semantics of the application error code carried in `RESET_STREAM` are defined by the application protocol. Only the application protocol is able to cause a stream to be terminated. A local instance of the application protocol uses a direct API call, and a remote instance uses the `STOP_SENDING` frame, which triggers an automatic `RESET_STREAM`.

Application protocols **SHOULD** define rules for handling streams that are prematurely canceled by either endpoint.

4 Error Codes

QUIC transport error codes and application error codes are 62-bit unsigned integers.

4.1 Transport Error Codes

This section lists the defined QUIC transport error codes that can be used in a CONNECTION_CLOSE frame with a type of 0x1c. These errors apply to the entire connection.

- **NO_ERROR (0x00)**: An endpoint uses this with CONNECTION_CLOSE to signal that the connection is being closed abruptly in the absence of any error.
- **INTERNAL_ERROR (0x01)**: The endpoint encountered an internal error and cannot continue with the connection.
- **CONNECTION_REFUSED (0x02)**: The server refused to accept a new connection.
- **FLOW_CONTROL_ERROR (0x03)**: An endpoint received more data than it permitted in its advertised data limits.
- **STREAM_LIMIT_ERROR (0x04)**: An endpoint received a frame for a stream identifier that exceeded its advertised stream limit for the corresponding stream type.

STREAM_STATE_ERROR (0x05): An endpoint received a frame for a stream that was not in a state that permitted that frame.

FINAL_SIZE_ERROR (0x06):

- An endpoint received a STREAM frame containing data that exceeded the previously established final size
 - an endpoint received a STREAM frame or a RESET_STREAM frame containing a final size that was lower than the size of stream data that was already received, or
 - an endpoint received a STREAM frame or a RESET_STREAM frame containing a different final size to the one already established.
- **FRAME_ENCODING_ERROR (0x07)**: An endpoint received a frame that was badly formatted – for instance, a frame of an unknown type or an ACK frame that has more acknowledgment ranges than the remainder of the packet could carry.
 - **TRANSPORT_PARAMETER_ERROR (0x08)**: An endpoint received transport parameters that were badly formatted, included an invalid value, omitted a mandatory transport parameter, included a forbidden transport parameter, or were otherwise in error.
 - **CONNECTION_ID_LIMIT_ERROR (0x09)**: The number of connection IDs provided by the peer exceeds the advertised active_connection_id_limit.
 - **PROTOCOL_VIOLATION (0x0a)**: An endpoint detected an error with protocol compliance that was not covered by more specific error codes.
 - **INVALID_TOKEN (0x0b)**: A server received a client Initial that contained an invalid Token field.

- **APPLICATION_ERROR (0x0c)**: The application or application protocol caused the connection to be closed.
- **CRYPTO_BUFFER_EXCEEDED (0x0d)**: An endpoint has received more data in CRYPTO frames than it can buffer.
- **KEY_UPDATE_ERROR (0x0e)**: An endpoint detected errors in performing key updates.
- **AEAD_LIMIT_REACHED (0x0f)**: An endpoint has reached the confidentiality or integrity limit for the AEAD algorithm used by the given connection.
- **NO_VIABLE_PATH (0x10)**: An endpoint has determined that the network path is incapable of supporting QUIC. An endpoint is unlikely to receive a CONNECTION_CLOSE frame carrying this code except when the path does not support a large enough MTU.
- **CRYPTO_ERROR (0x0100-0x01ff)**: The cryptographic handshake failed. A range of 256 values is reserved for carrying error codes specific to the cryptographic handshake that is used. Codes for errors occurring when TLS is used for the cryptographic handshake are described in *mini RFC: QUIC Handshake*.

In defining these error codes, several principles are applied. Error conditions that might require specific action on the part of a recipient are given unique codes. Errors that represent common conditions are given specific codes. Absent either of these conditions, error codes are used to identify a general function of the stack, like flow control or transport parameter handling. Finally, generic errors are provided for conditions where implementations are unable or unwilling to use more specific codes.

4.2 Application Protocol Error Codes

The management of application error codes is left to application protocols. Application protocol error codes are used for the RESET_STREAM frame, the STOP_SENDING frame, and the CONNECTION_CLOSE frame with a type of 0x1d.